# 2022-2023 Resit Exam

## CPT104 Operating Systems

Exam Answers & Explanations

Generated by OClaw

## Question 1 (40 marks)

Q: [6 marks] 1. If throughput is more important than response time, should the time quantum be large or small?

When throughput (completed tasks per unit time) is prioritized over response time:

Use a relatively LARGE time quantum.

Reasoning:
- Larger quantum = fewer context switches per unit time
- Fewer context switches = less overhead (saving/restoring state)
- Less overhead = more CPU time spent on actual process execution
- More execution time = higher throughput

Trade-off:
- Response time suffers (processes wait longer for their turn)
- But since throughput is the priority, this is acceptable
- Very large quantum approaches FCFS behavior (good for batch systems)

[CHN] 当吞吐量（单位时间完成任务数）优先于响应时间时：

使用较大的时间片。

原因：
- 较大时间片 = 单位时间内上下文切换次数更少
- 更少切换 = 更少开销（保存/恢复状态）
- 更少开销 = 更多CPU时间用于实际进程执行
- 更多执行时间 = 更高吞吐量

权衡：
- 响应时间变差（进程等待更久才轮到）
- 但由于吞吐量优先，这是可接受的
- 极大时间片接近FCFS行为（适合批处理系统）

Q: [12 marks] 2. Why does a process get blocked? Under what conditions can it regain the CPU?

Reasons a process gets BLOCKED:
(1) I/O wait - waiting for disk read/write, network data, keyboard input, or any I/O operation to complete
(2) Resource wait - needs a resource currently held by another process (locks, semaphores, files)
(3) Event wait - waiting for a signal, message from another process, or timer event

Conditions to regain CPU (move from blocked to ready):
(1) The I/O operation completes or the resource is released by the holding process
(2) The awaited event occurs (signal received, message arrives)
(3) A higher-priority process that was running finishes or gets blocked, freeing the CPU
(4) The OS scheduler selects it from the ready queue (scheduler's decision)

[CHN] 进程被阻塞的原因：
(1) I/O等待——等待磁盘读写、网络数据、键盘输入等I/O操作完成
(2) 资源等待——需要另一进程持有的资源（锁、信号量、文件）
(3) 事件等待——等待信号、来自另一进程的消息或定时器事件

恢复CPU的条件（从阻塞移到就绪）：
(1) I/O操作完成或资源被持有进程释放
(2) 等待的事件发生（收到信号、消息到达）
(3) 正在运行的更高优先级进程完成或被阻塞，释放CPU
(4) OS调度器从就绪队列中选择它

Q: [6 marks] 3. Proportional frame allocation: total=115 frames, P1=32 pages, P2=189 pages, P3=65 pages.

Total frames available: 115
Total pages: 32 + 189 + 65 = 286

Proportional allocation:
P1: 32/286 x 115 = 12.87 rounded to 13 frames (11%)
P2: 189/286 x 115 = 76.06 rounded to 76 frames (66%)
P3: 65/286 x 115 = 26.14 rounded to 26 frames (23%)

Check: 13 + 76 + 26 = 115 frames (all allocated)

Proportions: P1=11%, P2=66%, P3=23%

This ensures larger processes get more frames, which is generally fair and efficient.

[CHN] 可用总帧数：115
总页数：32+189+65=286

按比例分配：
P1: 32/286 x 115 = 12.87，取13帧（11%）
P2: 189/286 x 115 = 76.06，取76帧（66%）
P3: 65/286 x 115 = 26.14，取26帧（23%）

验证：13+76+26=115帧（全部分配）

比例：P1=11%, P2=66%, P3=23%

较大的进程获得更多帧，通常公平且高效。

Q: [6 marks] 4. How to protect files in a directory where only the leader should have full control?

Use Unix file permissions or Access Control Lists (ACL):

Unix Permission Model:
- Owner (leader): rwx (read, write, execute) = rwx------ (700)
- Group (team members): rw- (read, write, no execute) = rw-rw---- (660)
- Others: r-- (read only) = r--r--r-- (444)

Result with chmod:
- Leader (owner): Full control - read, write, execute, delete
- Group members: Can read and modify files, but CANNOT delete the directory or change permissions
- Others: Can only read files (read-only access)

Alternatively, ACL provides finer-grained control:
- setfacl can assign specific permissions to specific users/groups
- More flexible for complex permission requirements

[CHN] 使用Unix文件权限或访问控制列表（ACL）：

Unix权限模型：
- 所有者（组长）：rwx（读、写、执行）= rwx------ (700)
- 组（团队成员）：rw-（读写，不可执行）= rw-rw---- (660)
- 其他人：r--（只读）= r--r--r-- (444)

chmod结果：
- 组长（所有者）：完全控制——读写执行删除
- 组成员：可读写文件，但不能删除目录或更改权限
- 其他人：只能读文件（只读访问）

ACL方案提供更细粒度的控制，可为特定用户/组分配特定权限。

Q: [10 marks] 5. List five process management activities performed by the operating system.

(1) Process Scheduling:
Deciding which process runs next and for how long. Includes short-term (CPU), medium-term (memory), and long-term (admission) scheduling.

(2) Process Creation and Termination:
Allocating and freeing PCBs (Process Control Blocks), assigning PIDs, setting up initial resources, and cleaning up when processes exit.

(3) Process Synchronization:
Managing concurrent access to shared resources using semaphores, mutexes, monitors, and other synchronization primitives to prevent race conditions.

(4) Deadlock Handling:
Detecting, preventing, or recovering from deadlocks. Includes deadlock detection algorithms, resource allocation strategies, and process termination/recovery mechanisms.

(5) Memory Management for Processes:
Allocating and deallocating memory (using paging, segmentation), handling page faults, managing virtual memory, and ensuring process isolation.

[CHN] 操作系统执行的五个进程管理活动：

(1) 进程调度：
决定下一个运行的进程及运行时长。包括短程（CPU）、中程（内存）和长程（准入）调度。

(2) 进程创建与终止：
分配和释放PCB（进程控制块），分配PID，设置初始资源，进程退出时清理。

(3) 进程同步：
使用信号量、互斥锁、监视器等管理共享资源的并发访问，防止竞态条件。

(4) 死锁处理：
检测、预防或恢复死锁。包括死锁检测算法、资源分配策略和进程终止/恢复机制。

(5) 进程内存管理：
分配和释放内存（分页、分段），处理缺页，管理虚拟内存，确保进程隔离。

## Question 2 (36 marks)

Q: [6 marks] 1. LRU page replacement: ref=1 2 3 2 1 5 2 1 6 2 5 6 3 1 3 6 1 2 4 3, frames=3.

LRU (Least Recently Used) replaces the page that hasn't been used for the longest time.

Step-by-step trace (frames=3):

1: [1,-,-] FAULT | 2: [1,2,-] FAULT | 3: [1,2,3] FAULT
2: HIT | 1: HIT | 5: [2,3,5] FAULT
2: HIT | 1: [3,5,1] FAULT | 6: [5,1,6] FAULT
2: [1,6,2] FAULT | 5: [6,2,5] FAULT | 6: HIT
3: [2,5,3] FAULT | 1: [5,3,1] FAULT | 3: HIT
6: [3,1,6] FAULT | 1: HIT | 2: [1,6,2] FAULT
4: [6,2,4] FAULT | 3: [2,4,3] FAULT

Total page faults: 15

[CHN] LRU（最近最少使用）替换最长时间未使用的页面。

逐步追踪（3帧）：
1:缺页[1,-,-] | 2:缺页[1,2,-] | 3:缺页[1,2,3]
2:命中 | 1:命中 | 5:缺页[2,3,5]
2:命中 | 1:缺页[3,5,1] | 6:缺页[5,1,6]
2:缺页[1,6,2] | 5:缺页[6,2,5] | 6:命中
3:缺页[2,5,3] | 1:缺页[5,3,1] | 3:命中
6:缺页[3,1,6] | 1:命中 | 2:缺页[1,6,2]
4:缺页[6,2,4] | 3:缺页[2,4,3]

总缺页次数：15

Q: [6 marks] 2. TLB EAT: TLB=40ns, memory=100ns, hit=60%. What hit rate for EAT=170ns?

Current calculation:
TLB hit path: 40 + 100 = 140 ns
TLB miss path: 40 + 100 + 100 = 240 ns

Current EAT = 0.6 * 140 + 0.4 * 240 = 84 + 96 = 180 ns

For target EAT = 170 ns:
170 = h * 140 + (1 - h) * 240
170 = 140h + 240 - 240h
170 = 240 - 100h
100h = 240 - 170 = 70
h = 70 / 100 = 0.70

Required hit rate = 70%

[CHN] 当前计算：
TLB命中路径：40+100=140ns
TLB未命中路径：40+100+100=240ns

当前EAT = 0.6*140 + 0.4*240 = 84+96 = 180ns

目标EAT=170ns：
170 = h*140 + (1-h)*240
100h = 70
h = 70/100 = 0.70

所需命中率 = 70%

Q: [10 marks] 3. SRTF scheduling: P1(arr=0,burst=7), P2(arr=2,burst=4), P3(arr=3,burst=9), P4(arr=5,burst=10).

SRTF (Shortest Remaining Time First) - preemptive version of SJF:

Gantt Chart:
|P1(0-2)|P2(2-6)|P1(6-9)|P3(9-18)|P4(18-28)|

Detailed:
- t=0: P1 arrives (remaining=7). P1 runs.
- t=2: P2 arrives (remaining=4 < 7). P1 preempted. P2 runs.
- t=3: P3 arrives (remaining=9). P2 continues (4 < 9).
- t=5: P4 arrives (remaining=10). P2 continues (remaining=2, still shortest). P2 finishes at t=6.
- t=6: P1(rem=5), P3(rem=9), P4(rem=10). P1 shortest. P1 runs, finishes at t=9.
- t=9: P3(rem=9), P4(rem=10). P3 shortest. P3 runs, finishes at t=18.
- t=18: P4 runs, finishes at t=28.

Waiting times (start - arrival):
P1: 0 (arrived at 0, ran first) then 4 (resumed at 6, waited from 2-6) = total wait = 4
P2: 0 (arrived at 2, ran immediately) = 0
P3: 6 (arrived at 3, ran at 9) = 6
P4: 13 (arrived at 5, ran at 18) = 13

Average waiting time = (4 + 0 + 6 + 13) / 4 = 23 / 4 = 5.75

[CHN] SRTF（最短剩余时间优先）——SJF的抢占式版本：

甘特图：|P1(0-2)|P2(2-6)|P1(6-9)|P3(9-18)|P4(18-28)|

详解：
- t=0: P1到达(剩余7)，运行
- t=2: P2到达(剩余4<7)，P1被抢占，P2运行
- t=3: P3到达(剩余9)，P2继续(4<9)
- t=5: P4到达(剩余10)，P2继续(剩余2仍最短)，t=6完成
- t=6: P1(rem=5)最短，运行至t=9完成
- t=9: P3(rem=9)最短，运行至t=18完成
- t=18: P4运行至t=28完成

等待时间：P1=4, P2=0, P3=6, P4=13
平均等待时间 = (4+0+6+13)/4 = 23/4 = 5.75

Q: [6 marks] 4. FCFS disk scheduling: requests=25,90,135,50,190,60, head=100.

FCFS (First-Come-First-Served) services requests in arrival order:

Service order: 100 -> 25 -> 90 -> 135 -> 50 -> 190 -> 60

Movement calculation:
|100-25| = 75
|25-90| = 65
|90-135| = 45
|135-50| = 85
|50-190| = 140
|190-60| = 130

Total head movement = 75 + 65 + 45 + 85 + 140 + 130 = 540 cylinders

Note: FCFS is simple and fair but often results in high total head movement due to random access patterns.

[CHN] FCFS（先来先服务）按到达顺序服务：

服务顺序：100->25->90->135->50->190->60

移动量：
|100-25|=75, |25-90|=65, |90-135|=45
|135-50|=85, |50-190|=140, |190-60|=130

总磁头移动量 = 75+65+45+85+140+130 = 540柱面

注意：FCFS简单公平，但由于随机访问模式通常导致较高的总磁头移动量。

Q: [8 marks] 5. Best-fit allocation: Jobs A=57K, B=920K, C=50K, D=701K. Blocks: B1=900K, B2=910K, B3=200K, B4=300K.

Best-fit allocates each job to the smallest block that can hold it:

Job A (57K):
- Fits in B1(900K), B2(910K), B3(200K), B4(300K)
- Best fit = B3(200K) (smallest that fits)
- B3 remaining: 200-57 = 143K

Job B (920K):

- Does NOT fit in any block! (largest block is B2=910K < 920K)
- B must wait or be rejected

Job C (50K):
- Fits in B3(143K remaining), B1(900K), B2(910K), B4(300K)
- Best fit = B3(143K) (smallest with enough space)
- B3 remaining: 143-50 = 93K

Job D (701K):
- Fits in B1(900K), B2(910K)
- Best fit = B1(900K) (smallest that fits)
- B1 remaining: 900-701 = 199K

Result: A->B3(143K waste), B->CANNOT FIT, C->B3(93K waste), D->B1(199K waste)

[CHN] 最佳适配：将每个作业分配到能容纳它的最小块：

作业A(57K)：
- 适合B1(900K), B2(910K), B3(200K), B4(300K)
- 最佳=B3(200K)（最小能容纳的）
- B3剩余：143K

作业B(920K)：
- 无法放入任何块！（最大块B2=910K<920K）
- B必须等待或被拒绝

作业C(50K)：
- 适合B3(143K剩余), B1(900K), B2(910K), B4(300K)
- 最佳=B3(143K)
- B3剩余：93K

作业D(701K)：
- 适合B1(900K), B2(910K)
- 最佳=B1(900K)
- B1剩余：199K

结果：A->B3(143K浪费), B->无法放置, C->B3(93K浪费), D->B1(199K浪费)

## Question 3 (12 marks) - Banker's Algorithm

Q: [12 marks] Banker's Algorithm: Check system safety and whether P3's request (0,1,0,0) can be safely granted.

Steps:
1. Calculate Need matrix: Need = Max - Allocation for each process
2. Run safety algorithm with current Available resources
3. For P3's request (0,1,0,0):
   a) Check if Request <= Need[P3]: if no, reject (asking for more than declared)
   b) Check if Request <= Available: if no, P3 must wait (not enough resources)
   c) Temporarily allocate: Available = Available - Request, Allocation[P3] += Request, Need[P3] -= Request
   d) Run safety algorithm on the resulting state
   e) If safe: grant the request. If unsafe: rollback and P3 must wait.

Apply systematically to determine the outcome based on the given resource matrices.

[CHN] 解题步骤：

1. 计算Need矩阵：Need = Max - Allocation
2. 用当前Available资源运行安全算法
3. 对P3请求(0,1,0,0)：
   a) 检查Request<=Need[P3]：若否，拒绝
   b) 检查Request<=Available：若否，P3必须等待
   c) 临时分配：Available-=Request, Allocation[P3]+=Request, Need[P3]-=Request
   d) 对结果状态运行安全算法
   e) 若安全则批准；若不安全则回滚，P3等待

根据给定的资源矩阵系统地应用算法确定结果。

## Question 4 (12 marks) - POSIX Pipes

Q: 1. Describe how the given C program works with pipe(), fork(), read(), write().

The program demonstrates inter-process communication using pipes:

1. pipe(fd) creates a pipe with fd[0] for reading and fd[1] for writing
2. fork() creates a child process (duplicate of parent)
3. Parent process:
   - Closes the read end (fd[0]) - parent only writes
   - write(fd[1], message, length) sends 'Hello, pipe!' to the pipe
   - Closes the write end
   - Prints confirmation message
4. Child process:
   - Closes the write end (fd[1]) - child only reads
   - read(fd[0], buffer, size) receives the message from the pipe
   - Prints the received message and byte count
   - Closes the read end

The pipe provides a one-way communication channel from parent to child.

[CHN] 该程序使用管道演示进程间通信：

1. pipe(fd)创建管道：fd[0]读端，fd[1]写端
2. fork()创建子进程（父进程副本）
3. 父进程：
   - 关闭读端fd[0]——父进程只写
   - write(fd[1], message, length)发送'Hello, pipe!'到管道
   - 关闭写端
   - 打印确认消息
4. 子进程：
   - 关闭写端fd[1]——子进程只读
   - read(fd[0], buffer, size)从管道接收消息
   - 打印收到的消息和字节数
   - 关闭读端

管道提供从父进程到子进程的单向通信通道。

Q: 2. What is the program's output?

Output (two lines):

parent: wrote message to the pipe
child: read 14 bytes from the pipe: Hello, pipe!

Explanation:
- Parent writes first, prints its message
- Child reads the 14-byte message ('Hello, pipe!' = 13 chars + 1 null or 14 chars)
- Child prints the byte count and message content

Note: The order is deterministic because:
- Parent writes to pipe before child can read (pipe is small, write blocks until read)
- Both processes run concurrently but the pipe synchronizes them

[CHN] 输出（两行）：

parent: wrote message to the pipe
child: read 14 bytes from the pipe: Hello, pipe!

解释：
- 父进程先写，打印其消息
- 子进程读取14字节消息（'Hello, pipe!'）
- 子进程打印字节数和消息内容

注意：顺序是确定的，因为：
- 父进程先写入管道
- 两个进程并发运行，但管道同步了它们

Q: 3. How to implement bidirectional communication between parent and child?

Two approaches for bidirectional (two-way) communication:

Approach 1: Two unidirectional pipes
- Create pipe1 (parent->child) and pipe2 (child->parent)
- Parent writes to pipe1, reads from pipe2
- Child reads from pipe1, writes to pipe2
- Each direction has its own pipe, avoiding conflicts

Approach 2: Named pipes (FIFOs)
- Use mkfifo() to create named pipes in the filesystem
- Both processes can open named pipes by name
- Create two FIFOs: one for each direction
- More flexible - can connect unrelated processes

Important: A single pipe is ALWAYS unidirectional. Bidirectional communication always requires at least two channels.

[CHN] 双向通信的两种方法：

方法1：两个单向管道
- 创建pipe1(父->子)和pipe2(子->父)
- 父进程写pipe1，读pipe2
- 子进程读pipe1，写pipe2
- 每个方向有自己的管道，避免冲突

方法2：命名管道（FIFO）
- 使用mkfifo()在文件系统中创建命名管道
- 两个进程都可以按名称打开命名管道
- 创建两个FIFO：每个方向一个
- 更灵活——可连接无关的进程

重要：单个管道始终是单向的。双向通信至少需要两个通道。